

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Высшая школа программной инженерии

Курсовая работа
по предмету «Системы программирования»
Разработка веб-приложения на базе Django

Выполнил студент
гр. 5140904/50401



Е.И. Старунская

Проверял преподаватель:
Н. О. Степина

Санкт-Петербург
2025

Содержание

Описание.....	3
Модели.....	3
Функции представления в приложении Collects.....	4
OccasionViewSet.....	4
CollectViewSet.....	6
Функции представления в приложении Payment.....	7
PaymentViewSet.....	7
Сериализаторы.....	11
Сериализатор для модели Collect и Occasion.....	12
Сериализатор для модели Payment.....	14
Роутеры.....	15
Реализация отправки письма на почту.....	16
Тесты.....	16
Тесты для моделей.....	17
Тестирование PaymentViewSet.....	20
Как все работает.....	21
Вывод.....	22

Описание

Цель работы — создать приложение на базе Django и Django Rest Framework, которое позволяет управлять денежными сборами. Пользователь может создать сбор (collect). Внутри сбора он должен указать повод (occassion).

Авторизованные пользователи могут делать платежи. Авторизация происходит с помощью JWT-токенов.

При создании сбора или платежа пользователь получает сообщение на электронную почту.

Функционал:

- Управление групповыми денежными сборами
- Возможность создания, чтения, обновления и удаления сборов и платежей
- Аутентификация и авторизация с помощью JWT-токенов
- Асинхронная обработка задач, используя Celery, для повышения производительности
- Отправка писем на электронную почту при создании сбора/платежа
- Реализовано кэширование данных, возвращаемых GET-эндпоинтом

Модели

Создадим два приложения — payment и collects.

Приложение collects содержит две модели — Occasion и Collect.

Поля модели Occasion:

- name — название повода сбора

Поля модели Collect:

- author — автор сбора
- name — название сбора
- occasion — повод сбора — ForeignKey на модель Occasion

description — описание

- final_sum — сумма, которую необходимо собрать

- `completion_datetime` — дата завершения сбора

Приложение `payment` содержит модель `Payment`.

Поля модели `Payment`:

- `contributor` — пользователи, сделавший платеж
- `collect` — `ForeignKey` на модель `Collect`, сбор, к которому относится платеж
- `sum` — сумма платежа

Функции представления в приложении `Collects`

Функции представления будем делать, используя `ModelViewSet`.

`OccasionViewSet`

Устанавливаем `queryset`, сериализатор, класс разрешений. `OccasionViewSet` — содержит методы `create`, `update`, `list`, `destroy`. С помощью `OccasionViewSet` мы можем создавать, обновлять, удалять повод и смотреть список поводов.

```
class OccasionViewSet(viewsets.ModelViewSet): 4 usages
    """
    ViewSet для просмотра поводов сборов
    """
    queryset = Occasion.objects.all()
    serializer_class = OccasionSerializer
    permission_classes = (OwnerOrReadOnly,)

class OccasionViewSet(viewsets.ModelViewSet): 4 usages
    """
    ViewSet для просмотра поводов сборов
    """
    queryset = Occasion.objects.all()
    serializer_class = OccasionSerializer
    permission_classes = (OwnerOrReadOnly,)
```

Рисунок 1. `OccasionViewSet`

CollectViewSet

ViewSet для работы со сборами средств, предоставляет эндпоинты для управления сборами. Позволяет создать, обновить, удалить сбор и посмотреть список сборов.

Метод `get_queryset`

Переопределим `queryset`, чтобы для каждого сбора посчитать собранную сумму — `current_sum` и число уникальных участников, сделавших платежи. Для этого используем метод `annotate`. SQL запрос будет выглядеть следующим образом:

```
SELECT collect.id, collect.name, collect.description, collect.author_id,  
collect.occasion_id, collect.final_sum, collect.completion_datetime,  
SUM(payment.sum) as current_sum,  
COUNT(payment.contributor_id) AS contributor_count  
FROM collect
```

LEFT OUTER JOIN payment ON [collect.id](#) = payment.collect_id

Таким образом избегается проблема N+1 запроса к БД и сборы без платежей также попадут в результат с NULL.

```
def get_queryset(self):  
    """  
    Переопределяем queryset, чтобы для каждого сбора  
    посчитать собранную сумму и количество участников.  
    """  
    return Collect.objects.annotate(  
        # Создаём новое поле 'current_sum', считая сумму всех полей 'sum'  
        # у связанных платежей. 'payment' — это related_name у ForeignKey в модели Payment.  
        current_sum=Sum('payment__sum'),  
        # Считаем количество уникальных участников. distinct=True,  
        # чтобы один и тот же человек не посчитался дважды.  
        contributors_count=Count(*expressions: 'payment__contributor', distinct=True)  
    ).order_by('-created_at')
```

```
def get_queryset(self):
    """
    Переопределяем queryset, чтобы для каждого сбора
    посчитать собранную сумму и количество участников.
    """
    return Collect.objects.annotate(
        # Создаём новое поле 'current_sum', считая сумму всех полей 'sum'
        # у связанных платежей. 'payment' — это related_name у ForeignKey в модели Payment.
        current_sum=Sum('payment__sum'),
        # Считаем количество уникальных участников. distinct=True,
        # чтобы один и тот же человек не посчитался дважды.
        contributors_count=Count(*expressions: 'payment__contributor', distinct=True)
    ).order_by('-created_at')
```

Рисунок 2. Переопределение метода get_queryset

Метод get_permissions

Возвращает разрешение в зависимости от действия:

- List - посмотреть список сборов может каждый пользователь
- Retrieve - посмотреть один сбор может каждый пользователь
- Create - создать сбор может только авторизованный пользователь
- Update - обновить сбор может только авторизованный пользователь
- Destroy - удалить сбор может только автор сбора

```
def get_permissions(self):
    """
    Возвращает разрешения в зависимости от действия
    """
    action_permissions = {
        "list": (permissions.AllowAny(),),
        "retrieve": (permissions.AllowAny(),),
        "create": (permissions.IsAuthenticated(),),
        "update": (OwnerOrReadOnly(),),
        "partial_update": (OwnerOrReadOnly(),),
        "destroy": (OwnerOrReadOnly(),),
    }
    return action_permissions.get(self.action, super().get_permissions())
```

Рисунок 3. Разрешения

Метод list

Возвращает список сборов с применением кэширования. Кэширование состоит в том, что, если данные есть в кэше - возвращаем их. Если в кэше ничего нет, сохраняем ответ в кэш.

```
def list(self, request, *args, **kwargs):  
    """  
    Возвращает список сборов средств с применением кэширования  
    """  
    cache_key = "collects_list"  
    cached_data = cache.get(cache_key)  
    if cached_data:  
        return Response(cached_data)  
    response = super().list(request, *args: *args, **kwargs)  
    cache.set(cache_key, response.data, 60 * 15)  
    return response
```

Рисунок 4. Метод list

Метод perform_create

Данный метод создает сбор. User берется из request. Сохраняем его в качестве автора. Отправляем письмо на электронную почту о том, что сбор создан. И удаляем кэш, чтобы пользователь не получил неактуальные старые данные из кэша.

```
def perform_create(self, serializer):  
    """Создает сбор"""  
    user = self.request.user  
    collect_instance = serializer.save(author=user)  
    send_collect_confirmation_task.delay(  
        user_id=user.id,  
        collect_id=collect_instance.id  
    )  
    cache.delete("collects_list")
```

Рисунок 5. Метод perform_create

Метод perform_update обновляет информацию о сборе и также удаляет кэш при обновлении информации. Метод perform_destroy удаляет сбор и также удаляет кэш.

Функции представления в приложении Payment

PaymentViewSet

ViewSet для работы со платежами, предоставляет эндпоинты для управления платежами. Позволяет создать, обновить, удалить платеж и посмотреть список платежей.

Метод `get_permission` совпадает с этим методом в `CollectViewSet`. Метод `list` возвращает список платежей с применением кэширования.

```
def list(self, request, *args, **kwargs):  
    """  
    Возвращает список платежей с применением кэширования.  
    """  
    cache_key = "payments_list"  
    cached_data = cache.get(cache_key)  
    if cached_data:  
        return Response(cached_data)  
  
    response = super().list(request, *args, **kwargs)  
    cache.set(cache_key, response.data, 60 * 15)  
    return response
```

Рисунок 6. Метод list

Метод `perform_create` создает новый платеж и отправляет письмо по электронной почте и удаляет кэш.


```
def perform_create(self, serializer):
    """
    Создает новый платеж и отправляет email
    :param serializer: Сериализатор для сбора средств.
    """
    user = self.request.user
    payment = serializer.save(contributor=user)
    send_payment_confirmation_task.delay(self.request.user.id, payment.id)
    cache.delete("payments_list")
    cache.delete("collects_list")
```

Рисунок 7. Метод perform_create

Метод perform_update обновляет данные платежа и удаляет кэш. Метод perform_destroy удаляет платеж полностью и удаляет кэш.

Сериализаторы

Сериализатор для модели Collect и Occasion

Сериализатор преобразовывает данные JSON в объекты Django ORM, с которыми работает БД.

```
class OccasionSerializer(serializers.ModelSerializer):
    class Meta:
        model = Occasion
        fields = ("id", "name")
```

Рисунок 8. Сериализатор для модели Occasion

К сериализатору для модели Collect добавляем еще два поля. Их нет в модели. Эти два поля current_sum текущая собранная сумма и contributors_count число пользователей, сделавших платеж, считаются в CollectViewSet в методе

perform_create. Также проставим поля только для чтения - автор, который берется из request и created_at, которое проставляется автоматически при создании.

```
class CollectSerializer(serializers.ModelSerializer): 2 usages
    image = serializers.ImageField(required=False)
    current_sum = serializers.DecimalField(
        max_digits=12,
        decimal_places=2,
        read_only=True
    )
    contributors_count = serializers.IntegerField(read_only=True)
    class Meta:
        model = Collect
        fields = (
            "name",
            "author",
            "description",
            "occasion",
            "current_sum",
            "final_sum",
            "created_at",
            "completion_datetime",
            "contributors_count",
            "image",
        )
        read_only_fields = ("author", "created_at")
```

Рисунок 9. Сериализатор для модели Collect

Сериализатор для модели Payment

Contributor - является ссылкой на другую модель User - поэтому проставляем PrimaryKeyRelatedField. Само поле contributor берем из контекста.

```

class PaymentSerializer(serializers.ModelSerializer): 2 usages
    contributor = serializers.PrimaryKeyRelatedField(read_only=True)
    class Meta:
        model = Payment
        fields = (
            "collect",
            "sum",
            "created_at",
            "contributor",
        )

    def create(self, validated_data):
        validated_data["contributor"] = self.context["request"].user
        return super().create(validated_data)

```

Рисунок 10. Сериализатор для модели Payment

Роутеры

Определяем корневой [urls.py](#)

```

urlpatterns = [
    # Apps:
    path(route: 'main/', include(main_urls, namespace='main')),
    path(route: "", include("server.apps.collects.urls")),
    path(route: "", include("server.apps.payment.urls")),
    # Health checks:
    path(route: 'health/', include(health_urls)),
    # django-admin:
    path(route: 'admin/doc/', include(admin_docs_urls)),
    path(route: 'admin/', admin.site.urls),
]

```

Рисунок 11. Корневой urls



Подключаем urls приложения collects и payment и административную панель Django.

Также подключим swagger-документацию. Она будет доступна по адресу `http://127.0.0.1:8000/swagger/`

```
urlpatterns += [
    re_path(
        route: r'^swagger(?:<format>\.json|\.yaml)$',
        schema_view.without_ui(cache_timeout=0),
        name='schema-json',
    ),
```

Рисунок 12. Подключение swagger-автодокументации

POST

http://127.0.0.1:8000/auth/jwt/create/

Docs

Params

Authorization

Headers (13)

Body

Scripts

Settings

Headers

9 hidden

	Key	Value
☐	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0b2t1bi90eXBlljoYWN...
☐	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0b2t1bi90eXBlljoYWN...
☐	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1bi90eXBlljoYWNj...
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1bi90eXBlljoYWNj...
	Key	Value

127.0.0.1:8000

Payment Service

Спросить Алису AI

Swagger

http://127.0.0.1:8000/swagger/?format=openapi

Explore

Payment Service

[Base URL: 127.0.0.1:8000/]

http://127.0.0.1:8000/swagger/?format=openapi

Документация для приложения создания денежных сборов Payment Service

Contact the developer

BSD License

Schemes

HTTP

Django Login

Authorize

Filter by tag

auth

POST

/auth/jwt/create/

auth_jwt_create_create

POST

/auth/jwt/refresh/

auth_jwt_refresh_create

POST

/auth/jwt/verify/

auth_jwt_verify_create

GET

/auth/users/

auth_users_list

POST

/auth/users/

auth_users_create

POST

/auth/users/activation/

auth_users_activation

GET

/auth/users/me/

auth_users_me_read

Рисунок 13. Swagger-автодокументация

Регистрируем urls приложения collects. Роутер создает необходимый набор эндпоинтов. Только что созданный роутер сгенерирует два эндпоинта:

- collects/,
- collects/<int:pk>/.

Теперь через эти эндпоинты будут доступны любые операции с моделью:

- POST-запрос на collects/ создаст новый сбор.
- Запросы PUT, PATCH или DELETE к адресу collects/<pk>/ изменят или удалят существующую запись.
- GET-запрос на те же адреса вернёт список объектов или один объект.

```
7 router = DefaultRouter()
8
9
10 router.register( prefix: r"collects", CollectViewSet)
11 router.register( prefix: r"occasion", OccasionViewSet)
12
13 urlpatterns = [
14     path( route: "", include(router.urls)),
15 ]
16
```

Рисунок 14. Urls приложения collects

Аналогично происходит для occasion.

POST

▼

http://127.0.0.1:8000/payment/

Docs

Params

Authorization

Headers (13)

☐ none

☐ form-data

☐ x-www-form-urlencoded

```
1 {
2   "sum": 200,
3   "collect": "3"
4 }
```

Аналогичным образом регистрируется роутер для приложения payment.

```

router = DefaultRouter()

router.register(prefix: r"payment", PaymentViewSet)

urlpatterns = [
    path(route: "", include(router.urls)),
]

```

Рисунок 15. Urls приложения payment

Реализация отправки письма на почту

В конфиге необходимо прописать следующие поля:

```

EMAIL_HOST = smtp.yandex.ru # Адрес хоста эл. почты
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_USE_SSL = False
EMAIL_HOST_USER = default@yandex.ru
EMAIL_HOST_PASSWORD = password # Пароль почты, с которой
будут отправляться письма
DEFAULT_FROM_EMAIL = default@yandex.ru # Пароль почты, с
которой будут отправляться письма

```

Для работы я использовала свою яндекс почту. Необходимо зайти в раздел безопасности и паролей, скопировать пароль приложения и этот пароль вставить в переменную EMAIL_HOST_PASSWORD. Разрешить доступ SMTP/IMAP.

Устанавливаем настройки для celery. Celery будем использовать для отправки писем в фоновом режиме. В качестве брокера используется Redis.

Для начала нужно установить значение по умолчанию для среды DJANGO_SETTINGS_MODULE, чтобы Celery знала, как найти проект Django.

Затем мы создали экземпляр Celery с именем server и поместили в переменную app.

Затем мы загрузили значения конфигурации Celery из объекта настроек из django.conf. Мы использовали namespace=«CELERY» для предотвращения коллизий с другими настройками Django. Таким образом, все настройки конфигурации для Celery должны начинаться с префикса CELERY_.

Наконец, app.autodiscover_tasks() говорит Celery искать задания из приложений, определенных в settings.INSTALLED_APPS.

```
celery.py × payment/models.py payment/tests.py payment/serializers.py
1 > import ...
4
5
6 os.environ.setdefault( key: "DJANGO_SETTINGS_MODULE", value: "server.settings")
7 app = Celery("server")
8 app.config_from_object( obj: "django.conf.settings", namespace="CELERY")
9 app.autodiscover_tasks()
10
```

Рисунок 16. celery.py

Напишем задачу, которая отправляет письмо на почту при создании платежа.

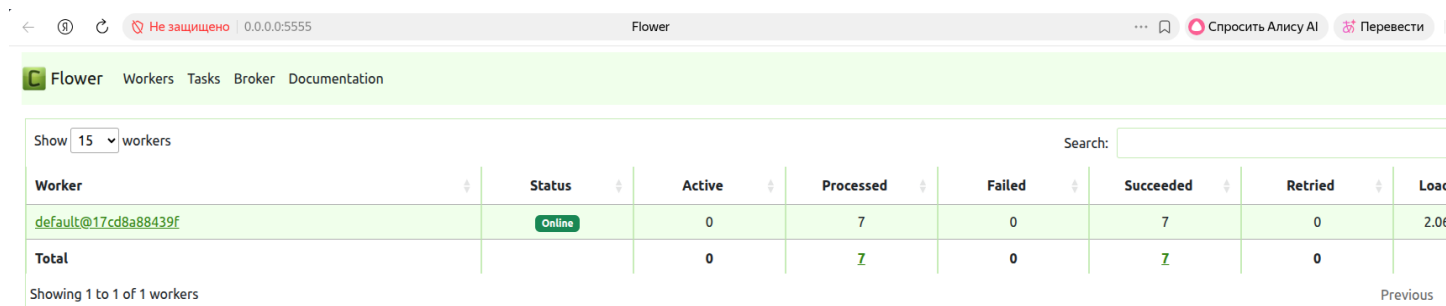
Фильтруем юзера по id, фильтруем нужный платеж. Создаем текст сообщения и отправляем. Чтобы celery поняла, что эта ее задача - ставим декоратор @shared_task. Аналогично пишется и задача отправки сообщения при создании

сбора.

```
16
17 @shared_task 2 usages
18 def send_payment_confirmation_task(user_id, payment_id):
19     """
20     Асинхронная задача отправки электронного сообщения.
21     Отправляет электронное сообщение на указанный адрес.
22
23     :param user_id: Id получателя.
24     :param payment_id: Id платежа.
25     """
26     user = User.objects.filter(id=user_id).first()
27     payment = Payment.objects.filter(id=payment_id).first()
28     if user and payment:
29         email_message = create_payment_confirmation_email(
30             user.username,
31             payment.collect.name,
32             payment.sum,
33         )
34         send_mail(
35             subject: "Payment Service: Групповые денежные сборы",
36             email_message,
37             DEFAULT_FROM_EMAIL, # Адрес почты, с которой будут отправляться письма
38             recipient_list: [user.email],
39             fail_silently=False,
40         )
41     else:
42         if not user:
43             logger.info("User not found", user_id=user_id)
44         if not payment:
45             logger.info("Payment not found", payment_id=payment_id)
```

Рисунок 17. Задача для отправки письма

Если открыть <http://0.0.0.0:5555> - то мы увидим очередь задач



Flower Workers Tasks Broker Documentation							
Show 15 workers	Search:						
Worker	Status	Active	Processed	Failed	Succeeded	Retried	Load
default@17cd8a88439f	Online	0	7	0	7	0	2.00
Total		0	7	0	7	0	
Showing 1 to 1 of 1 workers							
Previous							

Рисунок 18. Workers

←

Ⓢ

↺

Не защищено | 0.0.0.0:5555

Flower

...

🔖

Спросить Алису AI

🗨️ Перевести

Flower

Workers

Tasks

Broker

Documentation

Show

15

tasks

Search:

Name	UUID	State	args	kwargs	Result	Received	Started	Run
server.apps.payment.tasks.send_payment_confirmation_task	f63e5fed-5063-41df-ac12-0eb233737ff9	SUCCESS	(1, 4)	{}	None	2026-04-01 11:36:46.212	2026-04-01 11:36:46.213	0.
server.apps.payment.tasks.send_payment_confirmation_task	b533e034-bbc9-4ad8-b8eb-e6792eff7565	SUCCESS	(1, 3)	{}	None	2026-04-01 11:36:39.827	2026-04-01 11:36:39.829	0.
server.apps.payment.tasks.send_payment_confirmation_task	f8c462a6-00db-45c5-9131-b5d38c52b2e5	SUCCESS	(1, 2)	{}	None	2026-04-01 11:36:32.444	2026-04-01 11:36:32.445	1.
server.apps.payment.tasks.send_payment_confirmation_task	9e0f7221-019c-43d1-99ec-230a5cbd1d33	SUCCESS	(1, 1)	{}	None	2026-04-01 11:36:26.444	2026-04-01 11:36:26.444	1.
server.apps.collects.tasks.send_collect_confirmation_task	a097f043-4aa2-4525-b1e1-e7a52712550e	SUCCESS	()	{'user_id': 1, 'collect_id': 3}	None	2026-04-01 11:35:51.929	2026-04-01 11:35:51.930	0.
server.apps.collects.tasks.send_collect_confirmation_task	872471cf-920f-4d0d-bc34-a54acd87c1ed	SUCCESS	()	{'user_id': 1, 'collect_id': 2}	None	2026-04-01 11:35:16.778	2026-04-01 11:35:16.780	0.
server.apps.collects.tasks.send_collect_confirmation_task	221b08f8-9285-4e4f-aa7f-474f2ab05519	SUCCESS	()	{'user_id': 1, 'collect_id': 1}	None	2026-04-01 11:34:25.157	2026-04-01 11:34:25.158	1.

Showing 1 to 7 of 7 tasks

Previous

Рисунок 19. Задачи

Тесты

Тесты для моделей

Класс `TestPaymentModel` помечен декоратором `@mark.django_db`, который даёт тестам доступ к тестовой базе данных. Каждый тест выполняется в отдельной транзакции, которая автоматически откатывается после завершения — это гарантирует изоляцию тестов друг от друга. Класс содержит два тестовых метода, проверяющих структуру модели и корректность её строкового представления.

test_model_fields — проверка полей модели

Тест проверяет, что модель `Payment` содержит все необходимые поля. Через атрибут `Payment._meta.fields` получается список всех полей модели, из которого извлекаются их имена. Затем с помощью оператора `assert` последовательно проверяется наличие четырёх обязательных полей:

- `contributor` — внешний ключ на пользователя, совершившего платёж
- `collect` — внешний ключ на сбор, к которому относится платёж

- `sum` — сумма платежа
- `created_at` — дата и время создания платежа

Этот тест защищает от случайного переименования или удаления полей при рефакторинге модели.

`test_str_representation` — проверка строкового представления

Тест проверяет, что метод `__str__` модели `Payment` возвращает корректно отформатированную строку.

Для этого последовательно создаются все необходимые объекты в тестовой базе данных:

- Пользователь через `User.objects.create` с именем `test_user` и email `test@test.com`.
- Повод (`Occasion`) с названием "Test Occasion".
- Сбор (`Collect`) от имени созданного пользователя с суммой 1000 и датой окончания через 30 дней от текущего момента. Дата формируется динамически через `now() + timedelta(days=30)`, чтобы тест не зависел от конкретной даты запуска.
- Платёж (`Payment`) на сумму 100, связанный с созданным пользователем и сбором.

После создания платежа проверяется, что `str(payment)` возвращает строку в формате:

Платеж {id}. Платеж внес {contributor_id}. Сумма платежа {sum}. Дата платежа {created_at}. Сбор {collect_id}

Аналогичным образом выглядит тест для моделей `Occasion` и `Collect`

Тестирование PaymentViewSet

Класс `TestPaymentViewSet` наследуется от `APITestCase` — базового класса Django REST Framework для тестирования API-эндпоинтов. Класс содержит методы подготовки данных и восемь тестовых сценариев, покрывающих все основные операции с платежами.

Подготовка тестовых данных

Метод `setUpTestData` помечен декоратором `@classmethod` и выполняется один раз перед запуском всех тестов класса. В нём создаётся минимальный набор данных, необходимый для тестирования:

- Два пользователя — `user_1` (автор сбора и плательщик) и `user_2` (сторонний пользователь для проверки ограничений доступа). Оба созданы через `create_user`, который автоматически хеширует пароль.
- Повод для сбора (`Occasion`) с названием "Test Occasion".
- Сбор средств (`Collect`) от имени `user_1` с суммой 50 000 и датой окончания через 30 дней от момента запуска тестов.
- Платёж (`Payment`) на сумму 100 от `user_1` к созданному сбору.

Метод `setUp` выполняется перед каждым тестом и создаёт чистый экземпляр `APIClient` — HTTP-клиента для отправки тестовых запросов. Это гарантирует, что каждый тест начинается без авторизации.

Вспомогательный метод авторизации

Метод `get_authenticated_client` принимает объект пользователя, генерирует для него JWT access-токен через `AccessToken.for_user(user)` и устанавливает его в заголовок `Authorization: Bearer <token>`. Метод возвращает настроенный клиент, готовый к выполнению авторизованных запросов.

Тестовые сценарии

`test_payment_list` — проверяет, что список платежей (`GET /payments/`) доступен без авторизации. Ожидаемый статус ответа: 200 OK.

`test_payment_detail` — проверяет, что детальная информация об отдельном платеже (`GET /payments/{id}/`) доступна без авторизации. URL формируется динамически через `reverse("payment-detail")` с передачей первичного ключа платежа.

`test_create_payment_authenticated` — проверяет создание платежа авторизованным пользователем. Отправляется POST-запрос с данными: идентификатор сбора и сумма 50. Проверяется два условия: статус ответа 201 Created и фактическое наличие нового платежа в базе данных через `Payment.objects.filter(sum=50).exists()`.

`test_create_payment_unauthenticated` — проверяет, что неавторизованный пользователь не может создать платёж. Аналогичный POST-запрос без токена должен вернуть 401 Unauthorized.

`test_update_payment_by_author` — проверяет, что платёж нельзя изменить даже его автору. Отправляется PATCH-запрос с новой суммой 200. Ожидается 403 Forbidden. Дополнительно вызывается `refresh_from_db()` для обновления объекта из базы данных и проверяется, что сумма не изменилась.

`test_update_payment_by_another_user` — проверяет, что сторонний авторизованный пользователь (`user_2`) также не может изменить чужой платёж. Ожидается 403 Forbidden.

`test_delete_payment_by_another_user` — проверяет, что сторонний пользователь не может удалить чужой платёж. После DELETE-запроса проверяется, что платёж по-прежнему существует в базе данных.

`test_delete_payment_by_author` — проверяет, что даже автор платежа не может его удалить — операция удаления платежей запрещена для всех. Ожидается 403 Forbidden, запись в базе данных остаётся.

Как все работает

После запуска заходим в Postman. Для начала регистрируем пользователя.

На эндпоинт `http://127.0.0.1:8000/auth/users/` отправляем POST запрос. В теле запроса пишем `username`, `email`, `password`.

```
{  
  "email": "testuser@example.com",  
  "password": "StrongPassword123",  
  "username": "testuser"  
}
```

Код ответа должен быть 201.

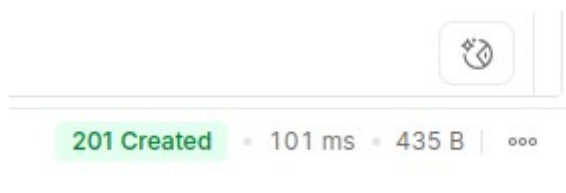


Рисунок 20. Код ответа при создании пользователя

Далее делаем авторизацию с помощью JWT-токенов. На эндпоинт `http://127.0.0.1:8000/auth/jwt/create/` отправляем в теле запроса `username`, `email`, `password`.

Пользователь получает `access` и `refresh` токены.



Рисунок 21. Токены для авторизации пользователя

Ассеесс токен мы помещаем в заголовок и делаем все остальные запросы вместе с токеном.

POST

▼

http://127.0.0.1:8000/auth/jwt/create/

Docs

Params

Authorization

Headers (13)

Body

Scripts

Settings

Headers

9 hidden

	Key	Value
☐	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0b2t1bI90eXBlljoiYWNj...
☐	Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ0b2t1bI90eXBlljoiYWNj...
☐	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1bI90eXBlljoiYWNj...
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ0b2t1bI90eXBlljoiYWNj...
	Key	Value

Рисунок 22.

Установка
токена в
Headers

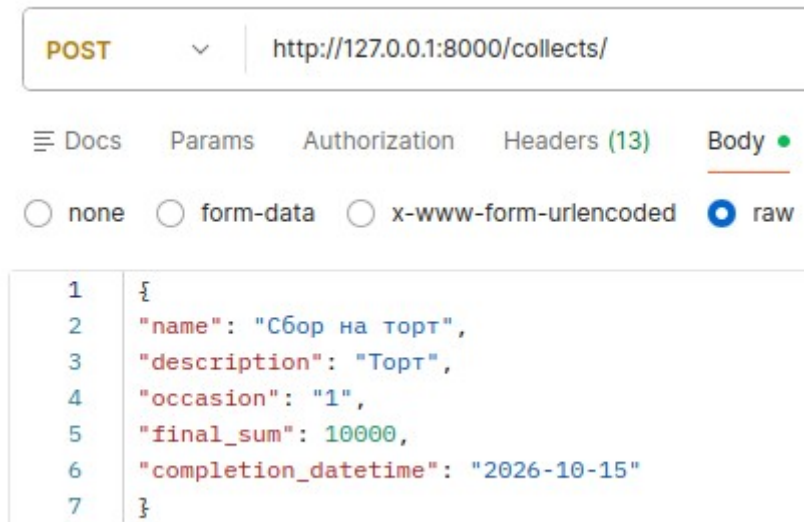
POST	http://127.0.0.1:8000/collects/			
Docs	Params	Authorization	Headers (13)	Body
☐ none	☐ form-data	☐ x-www-form-urlencoded	☒ raw	
<pre> 1 { 2 "name": "Сбор на торт", 3 "description": "Торт", 4 "occasion": "1", 5 "final_sum": 10000, 6 "completion_datetime": "2026-10-15" 7 }</pre>				

Создадим
occasion - повод
сбора. Для
этого на
эндпоинт

http://127.0.0.1:8000/occasion/ отправляем POST запрос. В теле нужно указать название повода:

```
{
  "name": "День Рождение"
}
```

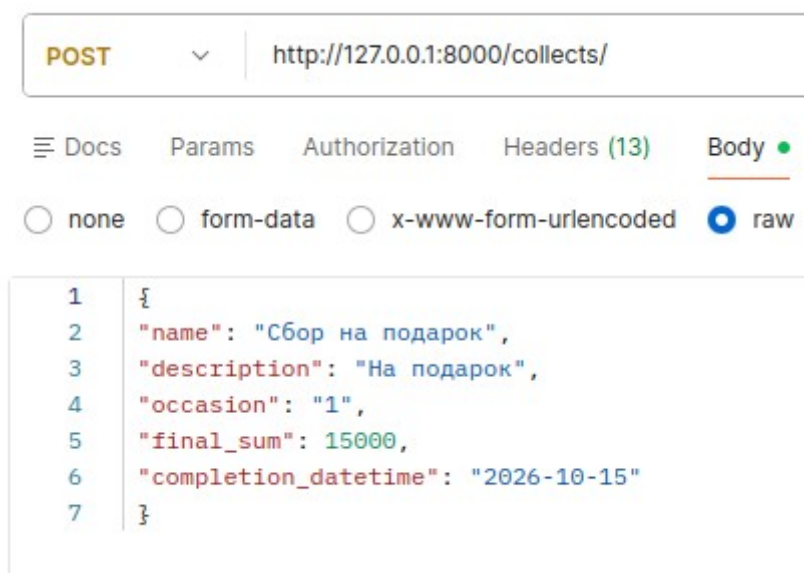
Далее мы можем создать сбор. На эндпоинт http://127.0.0.1:8000/collects/ отправляем POST запрос.



```
1 {
2   "name": "Сбор на торт",
3   "description": "Торт",
4   "occasion": "1",
5   "final_sum": 10000,
6   "completion_datetime": "2026-10-15"
7 }
```

Рисунок 23. Создание сбора

Создадим еще несколько сборов в рамках одного повода.



```
1 {
2   "name": "Сбор на подарок",
3   "description": "На подарок",
4   "occasion": "1",
5   "final_sum": 15000,
6   "completion_datetime": "2026-10-15"
7 }
```

Рисунок 24. Создание сбора

Код ответа должен быть 201 Created.

Создадим платеж. На эндпоинт `http://127.0.0.1:8000/payment/` отправляем POST-запрос с суммой платежа и `id` сбора.

The image shows a REST client interface. At the top, the HTTP method is set to **POST** and the URL is `http://127.0.0.1:8000/payment/`. Below this, there are tabs for **Docs**, **Params**, **Authorization**, and **Headers (13)**. Under the **Headers** tab, three radio buttons are visible: **none**, **form-data** (which is selected), and **x-www-form-urlencoded**. The main area displays a JSON body with the following content:

```
1 {  
2   "sum": 200,  
3   "collect": "3"  
4 }
```

Рисунок 25. Создание платежа

Создадим по аналогии еще несколько платежей.

Теперь посмотрим, что получилось. Отправим GET запрос на эндпоинт `http://127.0.0.1:8000/collects/`, чтобы получить все сборы.

HTTP <http://127.0.0.1:8000/collects/>

GET

<http://127.0.0.1:8000/collects/>

Docs

Params

Authorization

Headers (13)

Body ●

Scripts

Settings

Body

Cookies

Headers (11)

Test Results

{ }

JSON



Preview



Visualize



```
1
2   "count": 3,
3   "next": null,
4   "previous": null,
5   "results": [
6     {
7       "name": "Сбор на торт",
8       "author": 1,
9       "description": "Торт",
10      "occasion": 1,
11      "current_sum": "200.00",
12      "final_sum": "10000.00",
13      "created_at": "2026-04-01T11:35:51.901863Z",
14      "completion_datetime": "2026-10-15",
15      "contributors_count": 1
16    },
17    {
18      "name": "Сбор на шарик",
19      "author": 1,
20      "description": "Шарик",
21      "occasion": 1,
22      "current_sum": "500.00",
23      "final_sum": "5000.00",
24      "created_at": "2026-04-01T11:35:16.774830Z",
25      "completion_datetime": "2026-10-15",
26      "contributors_count": 1
27    },
28    {
29      "name": "Сбор на подарок",
30      "author": 1,
31      "description": "На подарок",
32      "occasion": 1,
33      "current_sum": "3000.00",
34      "final_sum": "15000.00",
35      "created_at": "2026-04-01T11:34:25.077989Z",
36      "completion_datetime": "2026-10-15",
37      "contributors_count": 1
38    }
39  ]
40 }
```

Рисунок 26. Сборы

Отправим GET запрос на эндпоинт <http://127.0.0.1:8000/payment/>, чтобы получить все платежи.

GET

▼

http://127.0.0.1:8000/payment/

≡ Docs

Params

Authorization

Headers (13)

Body ●

Scripts

Settings

Body

Cookies

Headers (11)

Test Results

{}

JSON

▼

▶ Preview

🖼 Visualize

▼

1

{

2

"count": 4,

3

"next": null,

4

"previous": null,

5

"results": [

6

{

7

"collect": 3,

8

"sum": "200.00",

9

"created_at": "2026-04-01T11:36:46.207025Z",

10

"contributor": 1

11

},

12

{

13

"collect": 2,

14

"sum": "500.00",

15

"created_at": "2026-04-01T11:36:39.822137Z",

16

"contributor": 1

17

},

18

{

19

"collect": 1,

20

"sum": "2000.00",

21

"created_at": "2026-04-01T11:36:32.439064Z",

22

"contributor": 1

23

},

24

{

25

"collect": 1,

26

"sum": "1000.00",

27

"created_at": "2026-04-01T11:36:26.438395Z",

28

"contributor": 1

29

}

30

]

31

}

Рисунок 27. Список платежей

Вывод

В рамках курсовой работы было разработано REST API веб-приложение на базе Django и Django REST Framework для управления групповыми денежными сборами.

В ходе работы были реализованы следующие компоненты:

- Модели данных — спроектированы и реализованы три взаимосвязанные модели: Occasion (повод сбора), Collect (сбор средств) и Payment (платёж).
- ViewSet-ы — реализованы OccasionViewSet, CollectViewSet и PaymentViewSet на основе ModelViewSet, с настроенными разрешениями для каждого действия (просмотр, создание, обновление, удаление).
- Оптимизация запросов — в методе get_queryset применён annotate для подсчёта текущей суммы сбора и числа участников в одном SQL-запросе, что позволило избежать проблемы N+1.
- Кэширование — реализовано кэширование ответов GET-эндпоинтов с автоматической инвалидацией кэша при создании, обновлении и удалении объектов.
- Асинхронная отправка писем — с помощью Celery и Redis в качестве брокера реализована фоновая отправка email-уведомлений пользователям при создании сбора или платежа.
- JWT-аутентификация — авторизация реализована с использованием django-rest-framework-simplejwt, пользователи получают access и refresh токены.
- Документация — подключена Swagger-автодокументация, доступная по адресу /swagger/.

POST http://127.0.0.1:8000/collects/

Docs Params Authorization Headers (13) Body

none form-data x-www-form-urlencoded raw

```

1 {
2   "name": "Сбор на шарики",
3   "description": "Шарики",
4   "occasion": "1",
5   "final_sum": 5000,
6   "completion_datetime": "2026-10-15"
7 }
```

Источники

1. Django REST Framework. API Guide

[Электронный ресурс]. — URL: <https://www.django-rest-framework.org/api-guide/> (дата обращения: 02.04.2026)

2. Django Documentation [Электронный ресурс]. — URL: <https://docs.djangoproject.com/en/5.2/> (дата обращения: 02.04.2026)
3. Celery. Introduction [Электронный ресурс]. — URL: <https://docs.celeryq.dev/en/stable/getting-started/introduction.html> (дата обращения: 02.04.2026).
4. Херман М. Асинхронные задания в Django с Celery [Электронный ресурс] // Habr : блог компании OTUS. — 2020. — URL: <https://habr.com/ru/companies/otus/articles/503380/> (дата обращения: 02.04.2026).

POST http://127.0.0.1:8000/collects/

Docs Params Authorization Headers (13) Body

none form-data x-www-form-urlencoded raw

```

1 {
2   "name": "Сбор на подарок",
3   "description": "На подарок",
4   "occasion": "1",
5   "final_sum": 15000,
6   "completion_datetime": "2026-10-15"
7 }
```

201 Created • 92 ms • 575 B | ...

 http://127.0.0.1:8000/collects/

GET http://127.0.0.1:8000/collects/

Docs Params Authorization Headers (13) Body • Scripts Settings

Body Cookies Headers (11) Test Results

{ } JSON Preview Visualize

```
1
2  "count": 3,
3  "next": null,
4  "previous": null,
5  "results": [
6    {
7      "name": "Сбор на торт",
8      "author": 1,
9      "description": "Торт",
10     "occasion": 1,
11     "current_sum": "200.00",
12     "final_sum": "10000.00",
13     "created_at": "2026-04-01T11:35:51.901863Z",
14     "completion_datetime": "2026-10-15",
15     "contributors_count": 1
16   },
17   {
18     "name": "Сбор на шарики",
19     "author": 1,
20     "description": "Шарики",
21     "occasion": 1,
22     "current_sum": "500.00",
23     "final_sum": "5000.00",
24     "created_at": "2026-04-01T11:35:16.774830Z",
25     "completion_datetime": "2026-10-15",
26     "contributors_count": 1
27   },
28   {
29     "name": "Сбор на подарок",
30     "author": 1,
31     "description": "На подарок",
32     "occasion": 1,
33     "current_sum": "3000.00",
34     "final_sum": "15000.00",
35     "created_at": "2026-04-01T11:34:25.077989Z",
36     "completion_datetime": "2026-10-15",
37     "contributors_count": 1
38   }
39 ]
```


 <http://127.0.0.1:8000/collects/>

GET  <http://127.0.0.1:8000/payment/>

 Docs Params Authorization Headers (13) Body ● Scripts Settings

Body Cookies Headers (11) Test Results

{ } JSON   Preview  Visualize 

```
1  {
2      "count": 4,
3      "next": null,
4      "previous": null,
5      "results": [
6          {
7              "collect": 3,
8              "sum": "200.00",
9              "created_at": "2026-04-01T11:36:46.207025Z",
10             "contributor": 1
11         },
12         {
13             "collect": 2,
14             "sum": "500.00",
15             "created_at": "2026-04-01T11:36:39.822137Z",
16             "contributor": 1
17         },
18         {
19             "collect": 1,
20             "sum": "2000.00",
21             "created_at": "2026-04-01T11:36:32.439064Z",
22             "contributor": 1
23         },
24         {
25             "collect": 1,
26             "sum": "1000.00",
27             "created_at": "2026-04-01T11:36:26.438395Z",
28             "contributor": 1
29         }
30     ]
31 }
```

